The Next Shape of the IT Business Capability Model

From Vendor Substrate to Owned Agentic Platforms

James D. Longmire
ORCID: 0009-0009-1383-7698
Correspondence: jdlongmire@outlook.com

Unaffiliated research. The views expressed are the author's own and do not represent any employer or institution.

Abstract

The IT business capability model has functioned for two decades as a procurement map. It enumerated the capabilities an enterprise required, and for each it named the commercial product that would supply it. The arrival of agentic artificial intelligence breaks the assumption beneath that map. Software is no longer a static tool that an enterprise rents and configures. It is becoming a learning system that encodes institutional knowledge as it operates, and that encoded knowledge compounds into durable advantage. The decisive architectural question is therefore no longer which vendor supplies a capability, but where the intelligence generated by that capability accrues. This paper names the pattern that answers the question deliberately. The pattern, here designated OAAD for its three constituent commitments to open-source software, agentic orchestration, and DevSecOps governance, reorganizes the capability model around owned substrate rather than rented substrate. The paper defines the pattern's essential elements, describes its runtime behavior, develops its sharpest application in sovereignty-constrained environments, states the consequences of adoption honestly, and distinguishes the pattern from its architectural neighbors. The claim is not that OAAD is a product to be purchased. The claim is that every enterprise of sufficient scale will decide where its agentic intelligence accrues, whether by deliberate architecture or by default, and that OAAD names the deliberate choice.

1. Introduction: The Naming Problem

Enterprises are adopting agentic platforms at speed, and they are doing so without a vocabulary for the most consequential architectural decision the adoption entails. The decision is rarely stated, because the prevailing capability model does not contain the terms in which it would be stated. That model treats software capabilities as commodities to be sourced. It asks, for each business function, which product best fills the slot. It does not ask what happens to the operational intelligence that the product accumulates while filling it, because under the assumptions of the model that intelligence did not exist. A customer relationship management system stored records. It did not learn the enterprise.

Agentic systems learn the enterprise. An agentic platform that triages incidents, drafts responses, routes approvals, and refines its own behavior against outcomes is accumulating a model of how the enterprise actually operates. That accumulation has value, and the value compounds. The question of where it compounds, inside the enterprise boundary or inside a vendor's substrate, is the question this paper concerns. It is an architectural question with strategic consequences, and at present most enterprises are answering it without realizing they have been asked.

The purpose of naming a pattern is to make a recurring decision visible and discussable. When a decision has no name, it is made by default, and defaults in enterprise architecture tend to favor whoever controls the substrate. The contribution of this paper is to name the pattern by which an enterprise retains the substrate, to define it with enough precision that an architect can recognize an instance of it and distinguish it from its neighbors, and to argue that the pattern describes a forced choice rather than an optional enhancement.

The paper proceeds as follows. Section 2 describes the traditional capability model and the forces now deforming it. Section 3 defines the OAAD pattern through its essential elements. Section 4 describes the pattern's runtime dynamics, where its governance commitments become concrete. Section 5 develops the sovereignty corollary, the pattern's application in environments that rented substrate cannot reach. Section 6 states the consequences of adoption, including its costs and its boundaries. Section 7 situates OAAD among related patterns. Section 8 concludes.

2. The Capability Model and the Forces Reshaping It

The IT business capability model is among the more durable artifacts of enterprise architecture practice. In its conventional form, documented across the major architecture frameworks, it decomposes an enterprise into the capabilities it must possess, organizes those capabilities into a navigable hierarchy, and serves as a shared reference against which investment, rationalization, and sourcing decisions are made (The Open Group, 2022; Business Architecture Guild, 2018). In practice the model has most often been used as a sourcing instrument. Each capability maps to a system, and each system maps to a vendor. The capability model becomes, in its operational use, a map of which commercial product owns which part of the enterprise.

This use rested on an assumption that held for as long as enterprise software was essentially static. A procurement map is adequate when the thing procured does not change in value through use. A licensed system delivered a fixed set of functions; the enterprise configured it, populated it with data, and operated it. The vendor's advantage lay in features and integration, and the

enterprise's exposure lay in switching costs and license fees. The capability model captured this arrangement faithfully because the arrangement was static enough to map.

The agentic shift dissolves the assumption. The clearest commercial demonstration of the shift is the emergence of agentic platforms that sit above the traditional system of record and orchestrate work across it (Salesforce, 2024). Such platforms are not better versions of the static system. They are a different kind of object. They observe, reason, act, and refine, and in doing so they encode the enterprise's operating patterns into the platform itself. The platform improves with use, and the improvement is specific to the enterprise that uses it. This is the property that the procurement map cannot represent, because the procurement map has no column for accrued intelligence.

A second force compounds the first. The major cloud providers are positioning their artificial intelligence layers not as point solutions but as universal orchestration fabrics spanning electronic mail, documents, meetings, code, and business data (Microsoft, 2025; Google Cloud, 2025; Amazon Web Services, 2025). Where the first force makes intelligence accrual possible, the second determines its destination. If the orchestration fabric belongs to the provider, then the intelligence accumulated through the enterprise's daily operation accrues to the provider's substrate. The enterprise's own operational learning strengthens a platform it does not own and cannot remove.

These forces stand in tension with one another, and the tension is what creates the architectural decision. Capability and ownership pull apart: the most capable agentic fabrics are the ones most tightly bound to a vendor's substrate. Velocity and control pull apart: the fastest path to agentic capability is to adopt the vendor's fabric wholesale, and that path concedes the substrate. Convenience and sovereignty pull apart: the convenient deployment is cloud-resident and continuously updated, and that deployment cannot reach environments where data residency and accreditation constraints forbid it.

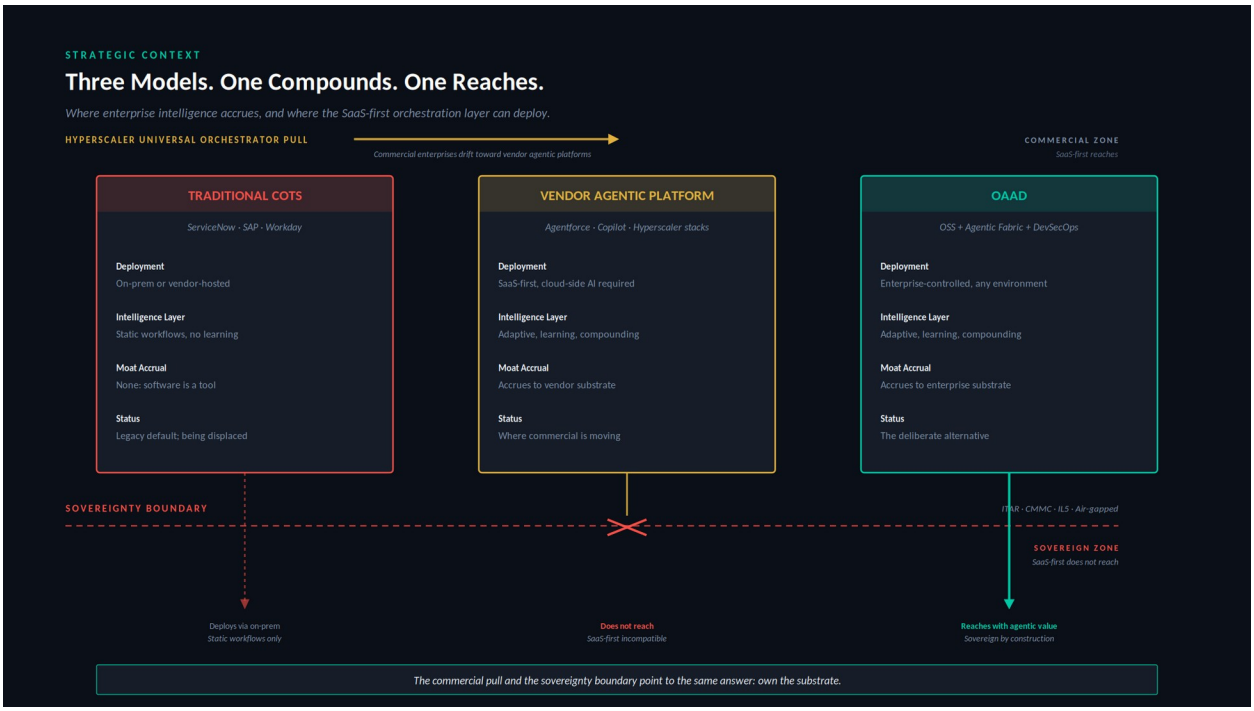

Figure 1. *Strategic Context. Three models of the capability stack, distinguished by where intelligence accrues and where the deployment model can reach.*

Figure 1 renders the resulting landscape. Three models now contend. The traditional commercial model persists as legacy, its software static and its intelligence accrual nil. The vendor agentic model represents the direction in which commercial enterprises are drifting under the pull of the universal orchestrators, its intelligence adaptive and compounding but accruing to the vendor. The third model, the subject of this paper, retains the compounding intelligence inside the enterprise boundary. The capability model is being redrawn by these forces whether or not architects participate in the redrawing. The remainder of this paper describes the shape it takes when the redrawing is deliberate.

The redrawing can be specified precisely. The conventional capability model recorded, for each capability, a small set of attributes adequate to a procurement decision: the capability itself, the system of record that supplied it, and the vendor on whom the enterprise consequently depended. Those attributes sufficed while software was static. They no longer suffice, because they cannot represent the decisions the agentic shift forces. The next shape of the model retains the procurement attributes and adds the dimensions the procurement map omitted. Table 1 lists the attributes of the extended schema and marks which carry forward from the procurement map and which the agentic shift adds.

Table 1. The capability model schema. Provenance marks which attributes carry forward from the procurement map, which are refined, and which the agentic shift adds.

Attribute	Provenance	What it records
Capability	Carried over	The business capability itself.
Current system of record	Carried over	The system supplying the capability today.
Residual vendor boundary	Refined	How much vendor dependency remains after the pattern is

Attribute	Provenance	What it records
		applied, and where it is irreducible.
Data ownership	New	Who owns and controls the data the capability generates and consumes.
Agentic automation potential	New	The degree to which the capability admits agentic orchestration rather than static operation.
Intelligence accrual location	***New***	***Where the operational intelligence the capability generates accumulates: vendor substrate or enterprise substrate. The decisive new attribute.***
Sovereignty requirement	New	The deployment constraint the capability is bound by: internet-reachable, enterprise-boundary, or sovereign.
Deterministic integration requirement	New	The transactional guarantees the capability requires beneath any agentic layer.
Governance and control baseline	New	The accreditation and control regime the capability must satisfy.
Substitution feasibility	New	Whether an owned open-source substrate can credibly supply the capability, and at what cost.

The first three rows are the procurement map, one of them refined to ask not merely which vendor supplies the capability but how much vendor dependency survives the pattern. The remaining rows are what the agentic shift adds. The single most important addition is the intelligence accrual location, because it is the attribute the procurement map structurally lacked and the attribute on which the pattern turns. An enterprise that completes this extended schema across its capability portfolio has performed the analysis the pattern requires, and the completed schema is itself the operating instrument by which the pattern is adopted capability by capability rather than wholesale. The remainder of the paper defines the pattern that the populated schema points toward.

3. The Pattern Defined

The pattern reorganizes the capability model around a single commitment: the enterprise owns the substrate on which its agentic intelligence accumulates. Everything else in the pattern follows from working out what that commitment requires. The designation OAAD names the three architectural commitments that together make the ownership commitment achievable. The first is open-source software as the business-capability substrate. The second is an agentic orchestration fabric operating above that substrate. The third is DevSecOps governance threaded through every layer rather than appended to the result. A capability stack that exhibits all three is an instance of the pattern. A stack that omits any one of them is something else, and the distinction matters, as Section 7 develops.

One qualification must be stated before the layers are described, because the pattern is easily misread without it. Open-source software is a necessary enabler of the pattern, not the pattern itself. Adopting open-source systems does not by itself produce ownership in the sense the pattern requires. Ownership here means operational control across the full stack: control of deployment, of model serving, of the data plane, of identity integration, of policy enforcement, of telemetry, of update cadence, of provenance, and of lifecycle risk. An enterprise can run entirely

open-source software and still surrender most of these controls to a managing vendor, in which case it has the license freedom of open source without the sovereignty the pattern is about. The open-source commitment removes the legal and architectural barriers to control; it does not exercise the control. The pattern is the exercise of control, and open source is what makes the exercise possible.

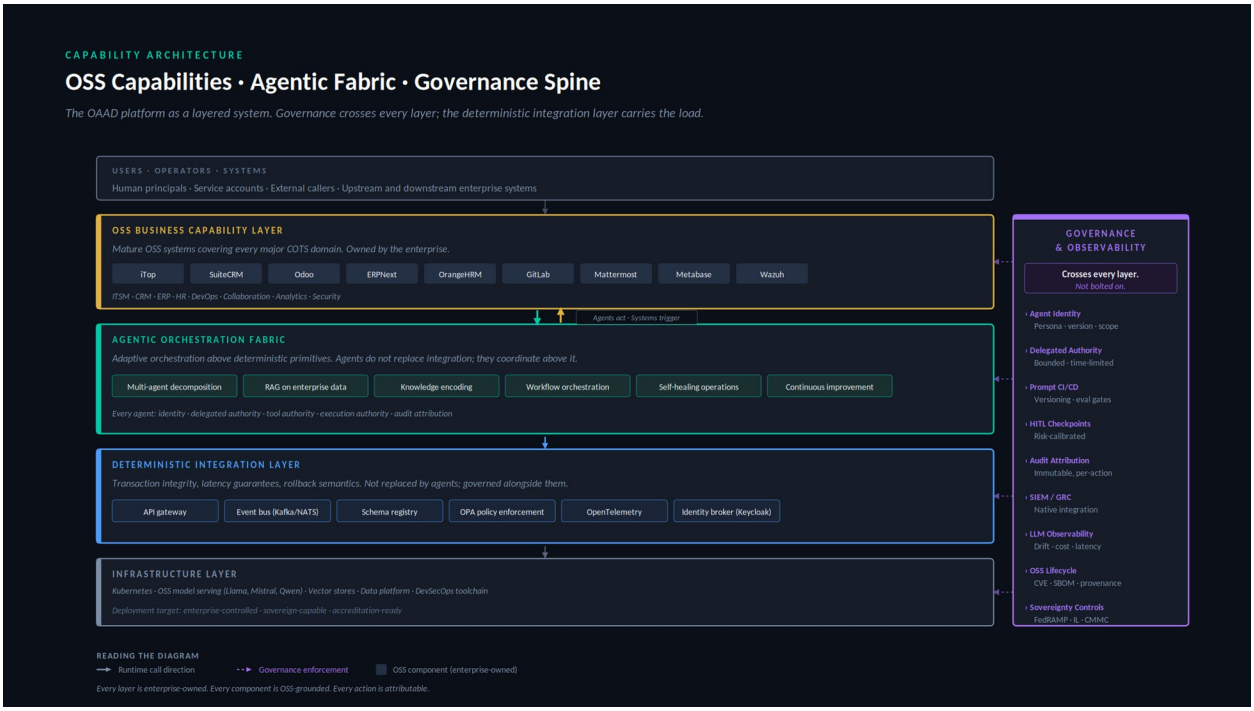

Figure 2. *Capability Architecture. The pattern as a layered system, with the governance spine crossing every layer and the deterministic integration layer carrying transactional load beneath the agentic fabric.*

Figure 2 presents the pattern as a layered architecture. The layers are described in turn.

The open-source business capability layer supplies the functional capabilities that the traditional model would have sourced from commercial vendors. Mature open-source systems now exist for every major capability domain that the conventional capability model enumerates: service management, customer relationship management, enterprise resource planning, human resources, software delivery, collaboration, analytics, and security operations. The essential property of this layer is not that the software is without license cost, though it generally is. The essential property is that the enterprise owns and operates it, and therefore owns whatever accrues to it. The software is not built from nothing; it is selected, deployed, and operated. Ownership in this pattern means operational control, not authorship.

The agentic orchestration fabric is the layer that distinguishes the pattern from a mere assembly of open-source systems. The fabric coordinates work across the capability systems, encodes institutional knowledge through retrieval over enterprise data, decomposes complex tasks across multiple agents, and operates routine functions without continuous human direction. It is the locus of the accruing intelligence. Because it sits above systems the enterprise owns and runs inside infrastructure the enterprise controls, the intelligence it accumulates accrues to the enterprise. This is the architectural mechanism by which the ownership commitment is honored in practice.

The deterministic integration layer sits beneath the fabric and carries the transactional load. It is a deliberate and essential element of the pattern, not an implementation detail. Agentic systems are probabilistic; enterprises require transactional integrity, latency guarantees, rollback semantics, and auditable contracts that probabilistic systems cannot themselves provide. The deterministic layer supplies these through conventional integration primitives: an interface gateway, an event backbone, a schema registry, policy enforcement, telemetry, and an identity broker. The agents act through this layer rather than around it. The pattern does not replace deterministic integration with agentic orchestration; it governs the two together, with the deterministic layer carrying the guarantees the agentic layer cannot.

The infrastructure layer carries the entire stack. It comprises the orchestration substrate, open-source model serving, vector stores, the data platform, and the delivery toolchain. Its defining requirement under the pattern is that it be enterprise-controlled and capable of deployment into constrained environments. The infrastructure layer is where the pattern's sovereignty properties are ultimately grounded, as Section 5 develops.

Crossing all of these is the governance spine. Its orientation in Figure 2, vertical rather than horizontal, is the figure's most deliberate choice. Governance in this pattern is not a layer that sits above or below the others. It is orthogonal to all of them, because every layer generates obligations that must be governed: agent identity and delegated authority in the fabric, policy enforcement and identity brokering in the integration layer, supply-chain provenance and lifecycle discipline in the open-source layer, control-baseline conformance in the infrastructure layer. A pattern that treated governance as a final layer to be added after the architecture was assembled would not be an instance of OAAD. The governance commitment is constitutive, and its orthogonality is what the spine depicts.

The pattern's essential elements can now be stated compactly. An instance of OAAD exhibits owned open-source capability systems, an agentic fabric that accrues intelligence to the enterprise, a deterministic integration layer that carries transactional guarantees, enterprise-controlled infrastructure capable of constrained deployment, and a governance discipline that crosses every layer. The presence of all five marks the pattern. The absence of any one marks something else.

4. Pattern Dynamics

A pattern is defined as much by how it behaves as by how it is composed. The static architecture of Section 3 acquires meaning only when one observes what happens when an agent runs. The dynamics also resolve a concern that the static description leaves open: an agentic fabric that accrues institutional intelligence is also a fabric that acts on enterprise systems, and the discipline by which those actions are authorized and recorded is the difference between a governable platform and an ungovernable one.

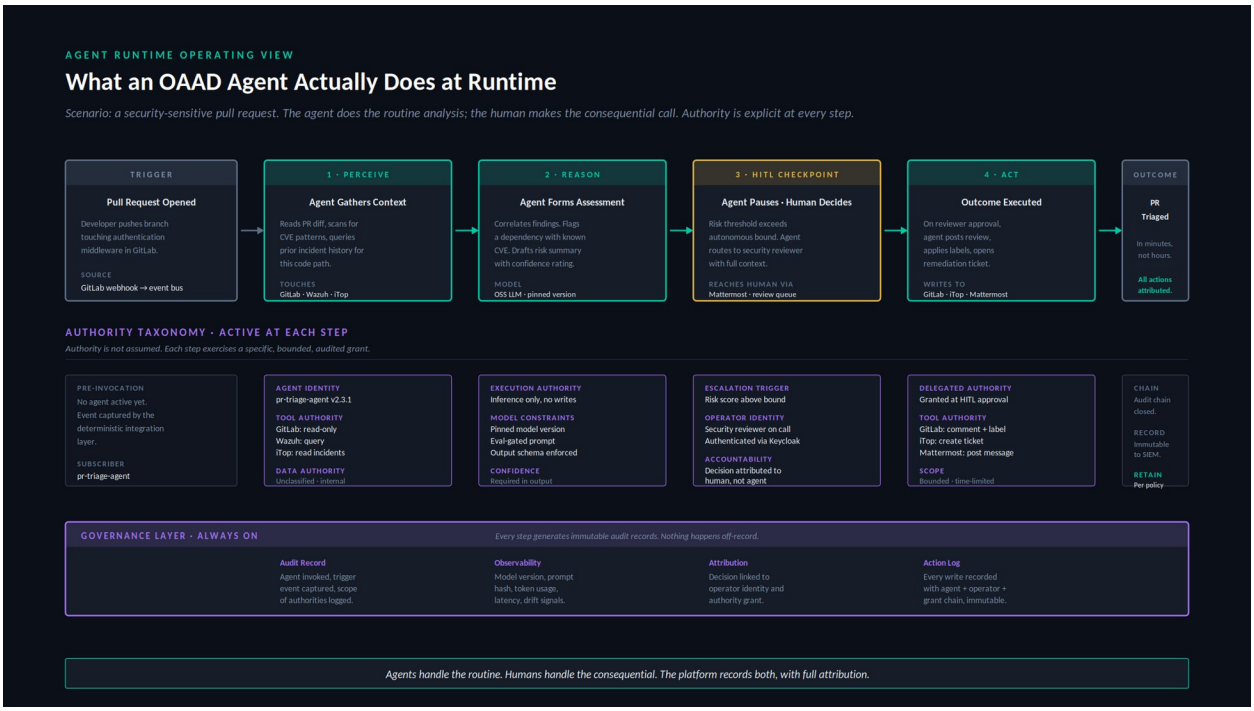

Figure 3. *Agent Runtime. A representative agent execution, showing the perceive-reason-checkpoint-act sequence, the authority exercised at each step, and the audit substrate that records throughout.*

Figure 3 traces a representative execution: an agent responding to a security-sensitive change in a software repository. The scenario is chosen because it exercises the full authority model and because it illustrates the pattern's central discipline, the division of labor between agent and human.

The execution proceeds through distinct phases. The agent perceives, gathering context across multiple systems under read-only authority. It reasons, forming an assessment under inference-only authority that permits no writes. At a checkpoint determined by policy, it pauses. The decision that follows, in this scenario the disposition of a security risk, exceeds the agent's autonomous bound, and so the agent routes the decision to a human reviewer rather than making it. The human decides. Only then, under authority freshly delegated at the moment of approval, does the agent act, writing its results back to the enterprise systems. The pattern's discipline is visible in the sequence: the agent performs the routine analysis that consumes human time to no good purpose, and the human performs the consequential judgment that ought not be delegated.

The authority exercised at each phase is explicit, bounded, and distinct. The pattern draws a taxonomy of authority that the conventional notion of a service account does not capture. Agent identity establishes who the agent is. Tool authority establishes which systems it may touch and in what mode. Execution authority establishes whether it may act or only infer. Delegated authority, granted at a human checkpoint, establishes what it may do once a human has approved. Data authority establishes which classifications of information it may process. These dimensions are independent, and treating them as independent is what allows an agent to hold read authority over a repository while holding no write authority, or to hold inference authority while holding no execution authority. An architecture that collapsed these dimensions into a single

grant would reproduce the governance failures that make ungoverned agentic systems hazardous.

Beneath the entire execution runs the governance substrate, recording continuously. Every phase generates immutable audit records: the trigger that invoked the agent and the scope of authority it carried, the model version and prompt that produced its reasoning, the human identity and the authority grant attached to the consequential decision, and the full chain of writes with their attribution. Nothing in the execution happens off the record. This is the property that makes the pattern accreditable, and it is the runtime expression of the governance commitment that Section 3 described as constitutive. A platform that acted without this substrate might accrue intelligence to the enterprise, but it would not be an instance of OAAD, because the governance commitment is part of the pattern's definition rather than an optional addition to it.

5. The Sovereignty Corollary

The pattern's sharpest application appears in sovereignty-constrained environments, where the difference between owning and renting the substrate stops being a matter of strategic preference and becomes visible in the deployment itself. It would be convenient for the argument to claim that the vendor agentic model simply cannot reach these environments. That claim was once nearly true and is now false, and the more interesting point survives its falsehood.

The hyperscalers have extended their agentic fabrics into accredited and even classified space. Microsoft made its agentic platform generally available in a government-community cloud built to meet FedRAMP High, ITAR, and CMMC requirements, with data held in domestic data centers operated by screened personnel (Microsoft, 2025). Amazon has operated air-gapped cloud regions accredited for classified workloads since 2014 and has committed to purpose-built classified AI infrastructure spanning all classification levels (Amazon Web Services, 2025). Google offers a distributed-cloud line for air-gapped workloads. The vendor model reaches the constrained zones. The reach objection, on which an earlier version of this argument would have rested, does not hold.

What the vendor reaches those zones with is the point. In every case the enterprise gains access to the constrained environment by renting a more isolated parcel of the vendor's substrate. The government-community cloud is operated by the vendor or its cleared personnel; the classified region is the vendor's region; the air-gapped distribution is the vendor's distribution. The deployment locus moves toward sovereignty while the substrate remains the vendor's. These variants also run on a distinct track, with model versions lagging the commercial offering and whole capabilities unavailable, which confirms that the sovereign deployment is a constrained instance of the vendor platform rather than the platform itself. The intelligence the enterprise's operation generates still accrues to the vendor's substrate, now sited in a sovereign data center rather than a commercial one. The question the capability model must answer, where the intelligence accrues, has the same answer in the classified region as in the public cloud.

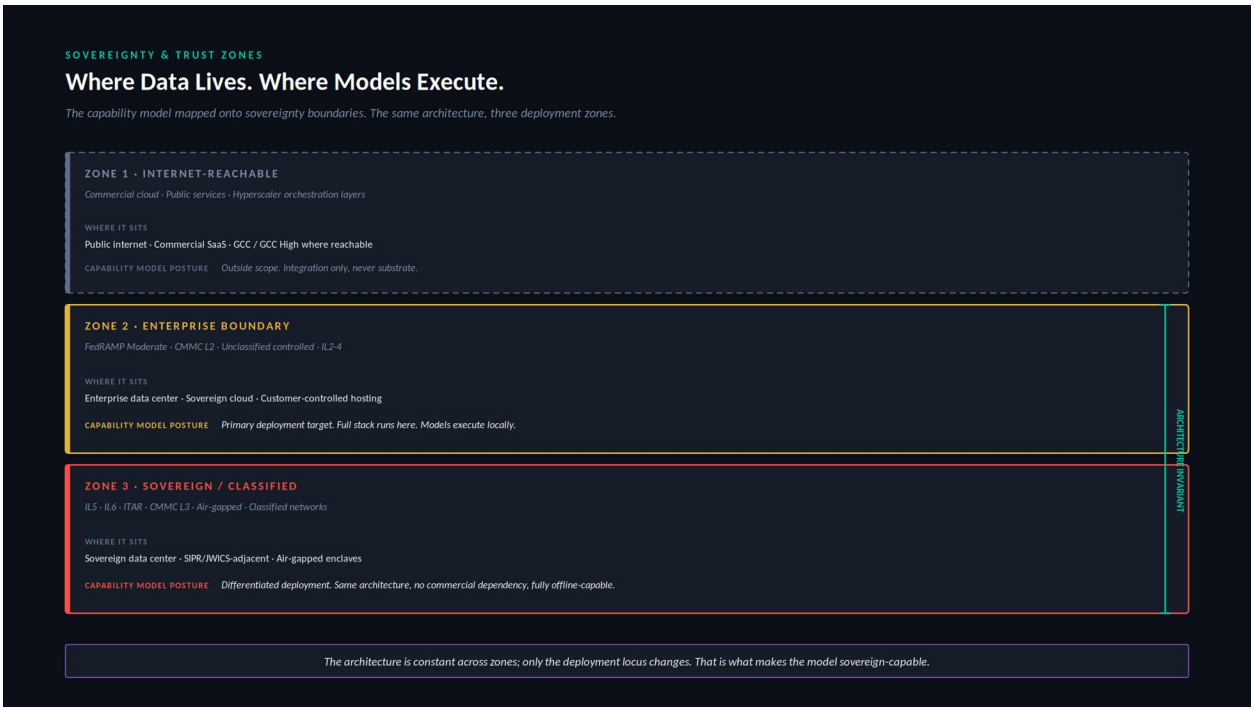

Figure 4. *Sovereignty and Trust Zones. The capability model mapped across three deployment zones, showing that the architecture remains invariant while the deployment locus changes.*

Figure 4 develops the corollary. It distinguishes three deployment zones. The internet-reachable zone is the native habitat of the vendor agentic model. The enterprise-boundary zone, governed by moderate accreditation baselines, is the pattern's primary deployment target, where the full stack runs and models execute locally. The sovereign zone, governed by the most stringent baselines and including air-gapped and classified environments, is where the pattern's properties become decisive. The vendor model now appears in all three zones, but it appears in the constrained zones as rented sovereign substrate, whereas the pattern appears there as owned substrate. That difference is what the figure is drawn to show.

The architectural claim that Figure 4 makes is the corollary's center. The pattern's component architecture is invariant across the three zones, and it is invariant in a specific sense the vendor model cannot reproduce. The same owned open-source capability systems, the same agentic fabric, the same governance spine, and the same integration primitives appear in each zone, and in each zone they remain the enterprise's. What changes from zone to zone is only the deployment locus. A model that executes on enterprise-controlled accelerators in a commercial data center executes on enterprise-controlled accelerators in an air-gapped enclave; it is the same open-weights model, owned and served by the enterprise, moved to a different location. The vendor model achieves a superficially similar portability, the same fabric appearing across zones, but it achieves it by relocating the enterprise's rental into successively more isolated parcels of the vendor's substrate. The fabric travels; the ownership does not.

The distinction is architectural rather than commercial, and it is the reason the vendor's portability is not the pattern's invariance. The vendor reaches a sovereign zone by standing up a sovereign instance of its own cloud, operated by the vendor under accreditation, and renting access to it. The enterprise that uses it has changed where its rental sits, not whether it rents. The pattern

reaches the same zone by deploying substrate the enterprise already owns into infrastructure the enterprise already controls. The sovereignty corollary thus follows directly from the ownership commitment that defines the pattern. An enterprise that owns its substrate can place that substrate wherever it controls infrastructure and retains ownership in every placement; an enterprise that rents its substrate can place it only where the lessor has built, and rents in every placement. Both can operate in the classified enclave. Only one owns what operates there.

The corollary generalizes beyond defense. Any enterprise subject to data-sovereignty regulation, to sectoral residency requirements, or to operational continuity mandates that constrain dependence on external services faces a constrained version of the same problem. The defense industrial base presents it in its most acute form, but the structure of the constraint, and the pattern's response to it, is general. The vendor model will follow its customers into these environments, because the commercial incentive to do so is strong. What it carries with it is the rental relationship. The pattern carries ownership. In the zones where it matters most whether the enterprise owns what learns from its operation, that is the difference that remains.

6. Consequences

A pattern is honest only if it states what its adoption costs as plainly as what it yields. The ownership commitment that defines OAAD creates control, and the same commitment creates responsibility. An enterprise that owns its substrate inherits obligations that a vendor previously absorbed. The pattern does not make these obligations disappear; it relocates them inside the enterprise boundary, where the enterprise must discharge them deliberately. This relocation is the pattern's central trade, and it should be entered with open eyes.

The first consequence is the lifecycle obligation. Open-source systems require disciplined lifecycle management: vulnerability monitoring, dependency auditing, provenance tracking, patch discipline, and a strategy for components whose maintenance lapses (National Telecommunications and Information Administration, 2021; Open Source Security Foundation, 2023). Under the vendor model these obligations were absorbed into the license and rendered invisible. Under the pattern they become first-class platform disciplines that the enterprise must staff and sustain. The obligation is not a defect of the pattern. It is the visible form of a cost that the vendor model concealed rather than eliminated.

The second consequence is the platform-team requirement. The pattern presupposes an enterprise willing to operate a platform rather than consume a product. This requires a standing team with the mandate, the budget, and the engineering depth to own the stack as a first-class enterprise asset. An enterprise unwilling to fund such a team should not adopt the pattern, because the pattern's benefits are inseparable from the operating commitment that produces them. The intelligence accrues to the enterprise precisely because the enterprise operates the platform; an enterprise that wishes to consume without operating has chosen the vendor model and should acknowledge the choice.

The third consequence is the economics of compute capacity. Serving open-weights models on enterprise-controlled infrastructure means provisioning and operating accelerator capacity that the vendor model rented by the token. This is a capital and operating commitment with its own

forecasting discipline: utilization planning, capacity headroom for peak agentic load, and the hardware refresh cycle that accelerator economics impose. The vendor model converted this into a usage-metered operating expense and made it invisible; the pattern makes it a visible line the enterprise must own. For some workloads the owned-capacity economics are favorable, particularly at sustained high utilization; for others, intermittent or spiky, they are not, and the schema of Table 1 should record where they are not.

The fourth consequence is the model evaluation burden. An enterprise that serves its own models inherits the obligation to evaluate them, an obligation the vendor model absorbed silently. Model selection, version pinning, regression testing against enterprise-specific tasks, and the evaluation gates that precede any promotion to production all become standing platform functions. The pattern's runtime discipline, described in Section 4, presupposes that the models behind it have been evaluated and that their behavior is characterized; producing and maintaining that characterization is work that does not appear in the vendor model because the vendor performed it out of view.

The fifth consequence is the operation of AI safety and adversarial testing. An agentic platform acting on enterprise systems is an attack surface and a source of operational risk, and the pattern relocates the responsibility for red-teaming, prompt-injection defense, output validation, and safety monitoring inside the enterprise boundary. This is not a one-time hardening exercise but a continuous operational function, analogous to security operations and properly integrated with it. The governance spine of Section 3 names the controls; this consequence names the team and the ongoing labor required to exercise them against an adversary who adapts.

The sixth consequence is open-source abandonment and fork risk. The pattern's substrate is composed of open-source projects, and open-source projects can stagnate, lose their maintainers, or change direction in ways that do not serve the enterprise. The pattern requires a deliberate response to this risk: monitoring the health of upstream projects, maintaining the capacity to fork and self-maintain a critical component if its upstream lapses, and weighing project vitality as a selection criterion alongside functional fit. The vendor model exchanged this risk for a different one, the risk of vendor lock-in and discontinuation; the pattern does not eliminate project risk but converts it into a form the enterprise can act on directly, since an owned fork is always available where an abandoned commercial product offers no such recourse.

The seventh consequence concerns the boundary of the pattern's applicability. The pattern does not claim to displace every category of enterprise system, and an honest definition names its limits. Systems whose value resides chiefly in vendor-held regulatory certifications, such as validated clinical and pharmaceutical manufacturing systems or accredited financial systems of record, are poor candidates for displacement, because in them the certification dominates the platform cost and the validation pathway is itself the product. Enterprise identity infrastructure is a genuine hard boundary, because every other system trusts the identity layer and its migration is a program of its own. The pattern integrates with these systems; it does not claim to replace them. A pattern that overreached its boundary would forfeit the credibility on which its core claims depend.

Against these costs stands the yield. The enterprise that adopts the pattern retains the intelligence its operations generate, accumulating a compounding asset that the vendor model would have

directed elsewhere. It retains the ability to deploy into environments that the vendor model cannot reach. It retains architectural control over a capability stack that would otherwise be governed by a lessor's roadmap. And it converts a recurring rent into an owned asset, trading a perpetual external dependency for an internal capability. Whether the yield justifies the cost is a determination each enterprise must make against its own circumstances. The pattern's contribution is to make the trade explicit, so that the determination can be made deliberately rather than by default.

What the Pattern Does Not Claim

A pattern stated without its limits invites misreading, and several misreadings of OAAD are predictable enough to forestall directly. The following are not implications of the pattern, and a reader who draws them has misunderstood it.

The pattern does not claim that an enterprise should replace every commercial system. It claims that the enterprise should know, for each capability, where its intelligence accrues, and decide deliberately. Many capabilities will remain on commercial systems after that decision, and the schema of Table 1 exists precisely to record which ones and why.

The pattern does not claim that all open-source software is enterprise-ready. Open-source maturity varies by domain and by project, and selection under the pattern is a discriminating exercise, not a presumption in favor of any system that carries an open-source license.

The pattern does not claim that agents should bypass deterministic integration. The opposite is constitutive: agents act through the deterministic layer, which carries the transactional guarantees the agentic layer cannot provide. An architecture in which agents act directly on systems of record without that layer is not an instance of the pattern.

The pattern does not claim that locally served models are automatically safer. Local execution changes where a model runs and who controls it; it does not by itself make the model's behavior safe. The safety and evaluation obligations described among the consequences above are the work that local control makes possible and necessary, not a property that local control confers for free.

The pattern does not claim that sovereignty follows from software choice alone. Sovereignty follows from disciplined operation across the full stack. An enterprise that adopts open-source substrate but operates it without the lifecycle, evaluation, safety, and governance disciplines the pattern requires has not achieved sovereignty; it has only acquired the materials from which sovereignty could be built.

7. Related Patterns

A named pattern earns its standing partly by its relation to the patterns around it. OAAD is not the first attempt to organize the capability stack, and situating it among its neighbors clarifies what it does and does not claim.

The vendor agentic platform is the pattern's nearest neighbor and its principal foil. The two share the agentic fabric and the compounding intelligence; they differ on the single axis of substrate

ownership. The vendor pattern sites the fabric on rented substrate and accrues the intelligence to the lessor; OAAD sites the fabric on owned substrate and accrues the intelligence to the enterprise. The patterns are otherwise so similar that the distinction is easy to miss, which is precisely why naming it matters. An enterprise can adopt the vendor pattern believing it has adopted something architecturally equivalent to OAAD, and discover only later that the equivalence held everywhere except in the one place that compounds.

The traditional commercial capability model is the legacy from which both agentic patterns descend. It shares with OAAD the assumption of enterprise operation, but it predates the agentic fabric and therefore accrues no intelligence at all. Its software is static; its capability model is the procurement map described in Section 2. OAAD is in one sense the traditional model brought forward into the agentic era, retaining its commitment to enterprise ownership while adding the fabric that the static model lacked.

OAAD also stands in instructive relation to patterns outside the agentic lineage. Data mesh decentralizes data ownership to domain teams and treats data as a product (Dehghani, 2020). It shares with OAAD a commitment to ownership and federation, and the two compose naturally, since the agentic fabric requires governed access to domain data that a mesh is well suited to provide. Zero trust reorganizes security around the principle that no actor is trusted by default (Rose et al., 2020). It shares with OAAD the orthogonality of governance to function, and the authority taxonomy of Section 4 can be read as an application of zero-trust principles to agentic actors. Platform engineering, the discipline of treating internal platforms as products with dedicated teams, supplies the operating-model vocabulary that the pattern's platform-team requirement presupposes. OAAD does not compete with these patterns. It composes with them, occupying the specific niche they leave open: the organization of the agentic capability stack around owned substrate.

What distinguishes OAAD from all of these is the conjunction it names. Open-source substrate is not novel. Agentic orchestration is not novel. Governed delivery is not novel. The pattern's contribution is to identify that the three together constitute the architecture of owned agentic intelligence, and that this architecture is the deliberate answer to a question every enterprise of scale will be forced to answer. The name marks the conjunction, so that the conjunction can be recognized, adopted, and reasoned about as a unit.

8. Conclusion

The IT business capability model is changing shape because the thing it models has changed character. Software that learns the enterprise cannot be mapped by an instrument designed for software that did not. The procurement map served faithfully while capabilities were static commodities; it fails silently now that capabilities accrue intelligence, because it has no way to represent where the intelligence goes. The capability model's next shape must add the dimension the procurement map lacked, and that dimension is ownership of the substrate on which intelligence accrues.

This paper has named the pattern that adds the dimension deliberately. OAAD organizes the capability stack around owned open-source substrate, an agentic fabric that accrues intelligence

to the enterprise, a deterministic integration layer that carries transactional guarantees, enterprise-controlled infrastructure that reaches into constrained environments, and a governance discipline that crosses every layer. The pattern is defined by the conjunction of these elements, behaves according to an explicit authority discipline at runtime, finds its sharpest application where rented substrate cannot reach, and carries honestly stated costs that an adopting enterprise must be prepared to bear.

The central claim is not that every enterprise should adopt the pattern. It is that every enterprise of sufficient scale will decide where its agentic intelligence accrues, and that the decision will be made whether or not it is recognized as a decision. An enterprise that adopts a vendor agentic fabric without deliberation has decided to accrue its intelligence to the vendor's substrate. An enterprise that does nothing has decided to let the default decide. The pattern named here is the deliberate alternative: the architecture an enterprise adopts when it has recognized the decision and chosen to retain what its own operation generates. Naming the pattern does not compel the choice. It makes the choice visible, which is the necessary first condition of making it well.

References

Amazon Web Services (2025) *Amazon Bedrock AgentCore is now generally available.* AWS Machine Learning Blog, 13 October. Available at: https://aws.amazon.com/blogs/machine-learning/amazon-bedrock-agentcore-is-now-generally-available/ (Accessed: 21 May 2026).

Business Architecture Guild (2018) *A Guide to the Business Architecture Body of Knowledge (BIZBOK Guide)*, Version 6.5. Business Architecture Guild.

Dehghani, Z. (2020) *Data Mesh Principles and Logical Architecture.* martinfowler.com, 3 December. Available at: https://martinfowler.com/articles/data-mesh-principles.html (Accessed: 21 May 2026).

Google Cloud (2025) *The new Gemini Enterprise: one platform for agent development.* Google Cloud Blog. Available at: https://cloud.google.com/blog/products/ai-machine-learning/the-new-gemini-enterprise-one-platform-for-agent-development (Accessed: 21 May 2026).

Microsoft (2025) *Microsoft 365 Copilot is now available in GCC-High.* Microsoft Community Hub, Public Sector Blog, 2 December. Available at: https://techcommunity.microsoft.com/blog/publicsectorblog/microsoft-365-copilot-is-now-available-in-gcc-high/4473310 (Accessed: 21 May 2026).

National Telecommunications and Information Administration (2021) *The Minimum Elements for a Software Bill of Materials (SBOM)*, pursuant to Executive Order 14028. U.S. Department of Commerce, July. Available at: https://www.ntia.gov/report/2021/minimum-elements-software-bill-materials-sbom (Accessed: 21 May 2026).

Open Source Security Foundation (2023) *Supply-chain Levels for Software Artifacts (SLSA).* Available at: https://slsa.dev/ (Accessed: 21 May 2026). SPDX is standardized as ISO/IEC 5962:2021 under the Linux Foundation; CycloneDX is an OWASP project.

Rose, S., Borchert, O., Mitchell, S. and Connelly, S. (2020) *Zero Trust Architecture.* NIST Special Publication 800-207. Gaithersburg, MD: National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-207.

Salesforce (2024) *Salesforce's Agentforce Is Here: Trusted, Autonomous AI Agents to Scale Your Workforce.* Salesforce Press Release, 29 October. Available at: https://www.salesforce.com/news/press-releases/2024/10/29/agentforce-general-availability-announcement/ (Accessed: 21 May 2026).

The Open Group (2022) *TOGAF Series Guide: Business Capability Planning.* The Open Group. Available at: https://pubs.opengroup.org/togaf-standard/business-architecture/business-capability-planning.html (Accessed: 21 May 2026).

A note on sources. The vendor references above are drawn from primary product announcements and documentation current as of May 2026. Vendor agentic platforms are evolving rapidly, and the specific capabilities and deployment options cited may change; the architectural relationships the paper draws from them are more stable than the product details. The observation in Section 3 that mature open-source systems exist across the major capability domains is a general industry observation, and the specific systems named in Figure 2 are independently verifiable rather than resting on a single source.